

FUNDAMENTALS OF WEB TECHNOLOGIES

The Web and the Browser

HTTP, the World Wide Web, DNS, Web Servers, and Web Browsers



Lecture 1



3 Hours



Live Demos Included



WELCOME

Why Start Here?



Everything else builds on this

HTML, CSS, JavaScript, frameworks, and APIs all travel over the protocols and infrastructure covered today.



You will debug faster

Understanding request/response, DNS, and caching lets you read a Network tab and know exactly what is happening.



The stack is decades old, and still current

HTTP/1.1 fundamentals from 1997 still describe most of the web in 2026 — this knowledge does not expire.

Learning objective: trace a URL from address bar to rendered page, explain every network hop along the way, and run your own diagnostic commands (curl, dig, browser DevTools) to debug it.



ROADMAP

Today's Agenda — 3 Hours, 6 Parts



The Internet & the Web

Internet vs Web, brief history, anatomy of a URL

20 min



DNS Deep Dive

Domain names, resolution process, record types, live lookups

30 min



Coffee / stretch break

10 min



The HTTP Protocol

Requests, responses, methods, status codes, headers

40 min



Web Servers

Role, static vs dynamic, live mini-server, edge platforms (Cloudflare)

30 min



Coffee / stretch break

10 min



Web Browsers

Rendering engine, DevTools tour, full request lifecycle

30 min



Synthesis & Wrap-up

End-to-end trace, security note, Q&A, lab assignment

15 min

PART 1 / 6

The Internet & the Web

What's the difference, and where did it all come from?

20 MINUTES





CONCEPT

Internet $\neq$ World Wide Web



The Internet

A global network of interconnected computer networks.

Infrastructure: cables, routers, satellites, protocols (TCP/IP) that move bits from A to B.

Existed before the Web — email, FTP, Telnet, Usenet all ran on it in the 1970s–80s.



The World Wide Web

A service that runs on top of the Internet.

A system of linked documents (pages) identified by URLs, transferred via HTTP, and rendered by browsers.

Invented in 1989 — one of many applications that use the Internet (alongside email, streaming, gaming).



MENTAL MODEL

A Simple Analogy



The Internet is the road network.

Highways, intersections, traffic rules — the physical and logical infrastructure that lets any vehicle reach any address.



The Web is the postal/delivery service that uses those roads.

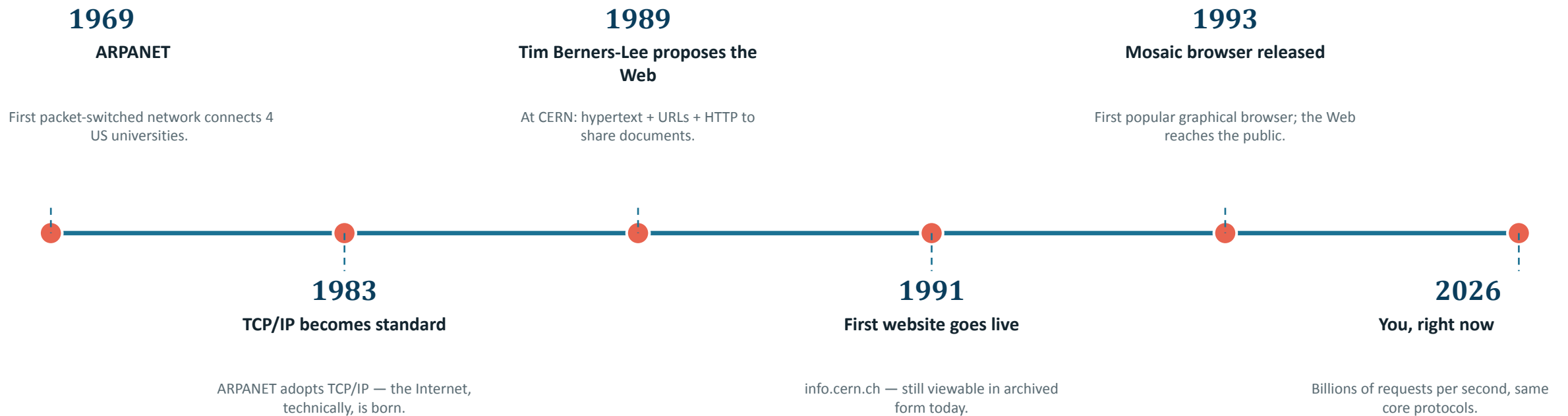
One specific way of using the roads — pick-up/drop-off rules (HTTP), addresses (URLs), and packages (HTML, images, JSON).

Other services share the same roads: email (SMTP), file transfer (FTP), video calls (RTP), online games — all use the Internet, none of them are "the Web."



CONTEXT

A Very Short History





CORE CONCEPT

Anatomy of a URL

```
https://www.example.com:443/products/shoes?color=red&size=42#reviews
```

`https://`

Scheme / Protocol

How to talk to the server (HTTP, HTTPS, FTP, mailto...)

`www.example.com`

Host / Domain

Which server to talk to — resolved via DNS

`:443`

Port

Which "door" on the server (443 = HTTPS default, often hidden)

`/products/shoes`

Path

Which resource on that server

`?color=red&size=42`

Query string

Extra parameters, key=value pairs joined by &

`#reviews`

Fragment

A location within the page — never sent to the server



LIVE DEMO

Picking Apart a Real URL



```
Open: https://en.wikipedia.org/wiki/HTTP#History
```

Do this together, in the browser:

- 1 Click into the address bar — point out scheme, host, path, fragment
- 2 Change the path to /wiki/DNS and hit Enter — same site, different resource
- 3 Append ?action=history — note nothing visibly changes, but it's sent to the server
- 4 Open DevTools (F12) → Network tab → reload → click the first request
- 5 Show the request URL is broken into the exact same components in the panel



CHECK YOUR UNDERSTANDING

Quick Discussion



If the Internet went down for one ISP in your city, would email between two people on a different ISP still work? Why?



Is streaming video on the Web? Is a video call on the Web? What's the test you'd apply?



Why do you think the fragment (#reviews) is never sent to the server — what would it even mean for a server to receive it?

PART 2 / 6

DNS Deep Dive

How a name like example.com becomes an IP address

30 MINUTES





CONCEPT

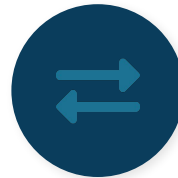
Why DNS Exists

Computers address each other by IP

```
93.184.216.34  
2606:2800:220:1:248:1893:25c8:1946
```

Humans remember names

```
example.com
```



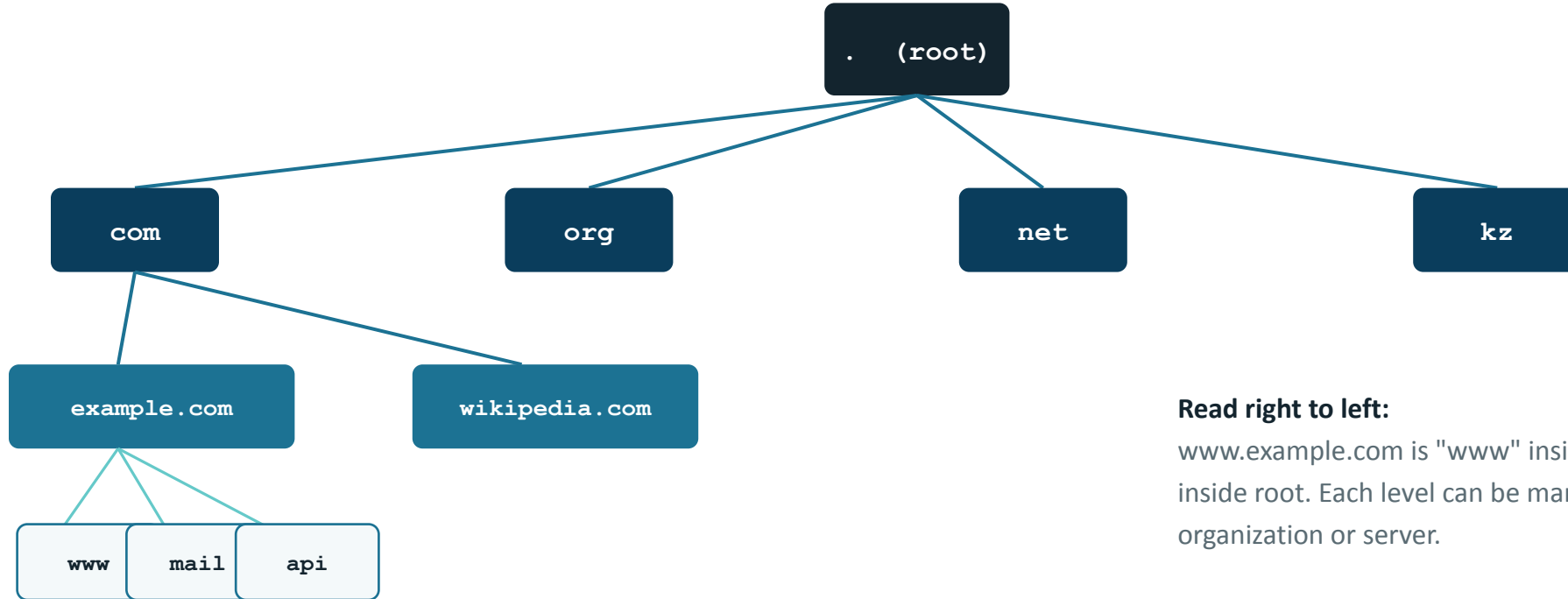
DNS is the directory service that translates between the two — the "phonebook of the Internet."

Without DNS: every link, bookmark, and business card would need to carry raw IP addresses — and those addresses can change.



STRUCTURE

The DNS Namespace Is a Tree



Read right to left:

`www.example.com` is "www" inside "example" inside ".com" inside root. Each level can be managed by a different organization or server.



PROCESS

The Resolution Journey, Step by Step

1

Browser cache

Have I looked this up recently? Check local cache first.



2

OS / resolver cache

Operating system keeps its own cache of recent lookups.



3

Recursive resolver

Usually your ISP or 8.8.8.8 / 1.1.1.1 — does the legwork for you.

4

Root server (.)

"I don't know — ask the .com server" (returns a referral).



5

TLD server (.com)

"I don't know the IP — ask example.com's nameserver".



6

Authoritative NS

The source of truth: "www.example.com = 93.184.216.34".

If any cache already has the answer, the journey stops early — that's why repeat visits to a site resolve almost instantly.



DETAIL

Recursive Queries & Caching (TTL)

Recursive resolver does the walking

Your device asks

one question

to a recursive resolver (e.g. 1.1.1.1).

The resolver makes multiple "iterative" queries on your behalf — root, then TLD, then authoritative — and returns one final answer.

TTL: Time To Live

Every DNS record has a TTL in seconds (e.g. 300, 3600, 86400).

Resolvers cache the answer for that long, then must ask again.

Low TTL = flexible, more lookups. High TTL = faster, slower to update (relevant before migrations!).

Common DNS Record Types

A Hostname → IPv4 address

CNAME Alias — hostname → another hostname

TXT Free-text — domain verification, SPF/DKIM

AAAA Hostname → IPv6 address

MX Mail exchange — where to deliver email

NS Which nameservers are authoritative



LIVE DEMO

Resolving a Domain From the Terminal



```
$ dig example.com +short
93.184.216.34

$ dig example.com MX +short
(often empty for example.com)

$ dig +trace example.com
...shows root -> .com -> auth NS
```



```
> nslookup example.com
Server:    1.1.1.1
Address:   93.184.216.34

$ nslookup -type=NS example.com
...lists authoritative nameservers
```

Try it yourselves (replace example.com with your own site):



```
$ dig yourdomain.com A +short      # IPv4 address
$ dig yourdomain.com AAAA +short   # IPv6 address
$ dig yourdomain.com NS +short     # who's authoritative
$ ping -c 4 yourdomain.com         # confirms reachability
```



PRACTICAL

Common DNS Pitfalls You'll Hit



"It works on my machine"

Stale local cache — try `ipconfig /flushdns` (Windows) or `sudo dscacheutil -flushcache` (macOS).



Site unreachable right after a change

TTL hasn't expired everywhere yet — propagation can take from minutes to 48 hours.



NXDOMAIN error

The domain genuinely doesn't exist, or was typed wrong, or hasn't propagated yet.



Works on WiFi, fails on mobile data

Different networks use different resolvers — useful for isolating DNS vs server issues.

The HTTP Protocol

The language browsers and servers speak to each other

40 MINUTES



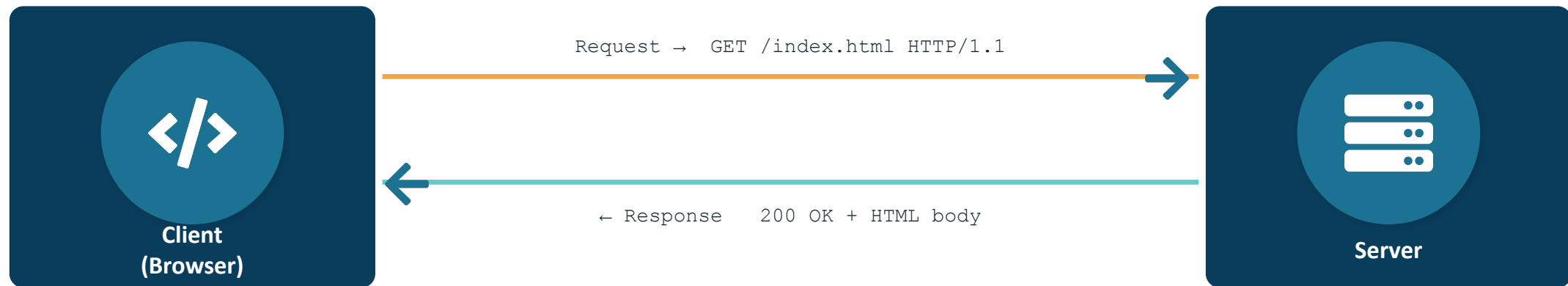
HTTP



CONCEPT

HTTP: HyperText Transfer Protocol

A request–response protocol. The client (browser) sends a request; the server sends back a response. That's the entire model.



Key property: HTTP is stateless.

Each request is independent — the server doesn't inherently remember previous requests from the same client. (We'll see how cookies work around this.)



Anatomy of an HTTP Request

```
GET /products/shoes?color=red HTTP/1.1
```

```
Host: www.example.com
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
Accept: text/html,application/xhtml+xml
```

```
Accept-Language: en-US,en;q=0.9
```

```
Cookie: session_id=alb2c3; theme=dark
```

```
Connection: keep-alive
```

(blank line separates headers from body)

(body – present for POST/PUT, empty here for GET)

Request line: Method, path, HTTP version
What to do, what to get, what to accept, cookies

Blank (optional): Data being used (forms, JSON, files)



Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
```

```
Date: Sun, 21 Jun 2026 10:15:00 GMT
```

```
Content-Type: text/html; charset=UTF-8
```

```
Content-Length: 1256
```

```
Cache-Control: max-age=3600
```

```
Set-Cookie: session_id=a1b2c3; HttpOnly
```

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head><title>Shoes</title></head>
```

```
  <body>...</body>
```

```
</html>
```

Status line: HTTP version + status code + reason phrase **Headers:** metadata about the response **Body:** the actual content (HTML here)



VOCABULARY

HTTP Methods (Verbs)

GET

Retrieve a resource

No body. Safe & idempotent. Can be cached.

POST

Create / submit data

Has a body. Not idempotent — repeating it may create duplicates.

PUT

Replace a resource entirely

Has a body. Idempotent — repeating it gives the same result.

PATCH

Partially update a resource

Has a body. Only changes specified fields.

DELETE

Remove a resource

Usually no body. Idempotent.

HEAD

Like GET, headers only

No body returned — useful for checking existence/size.



HTTP Status Codes

1xx

Informational

Request received, continuing process

100 Continue

2xx

Success

The request worked

200 OK · 201 Created · 204 No Content

3xx

Redirection

Go look somewhere else

301 Moved Permanently · 302 Found · 304 Not Modified

4xx

Client Error

You (the client) made a mistake

400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found

5xx

Server Error

The server messed up

500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable



DETAIL

Headers Worth Knowing

Content-Type

What kind of data is in the body (text/html, application/json, image/png)

Cache-Control

How long, and whether, to cache this response

Authorization

Credentials — e.g. Bearer tokens, Basic auth

Location

Used with redirects (3xx) — where to go instead

Content-Length

Size of the body in bytes

Cookie / Set-Cookie

How servers maintain state across stateless requests

User-Agent

Identifies the client software making the request

Referer

What page linked to this request (note the historic typo!)



EVOLUTION

HTTP Versions & HTTPS

HTTP/1.1

1997

One request per connection at a time (head-of-line blocking); text-based.

HTTP/2

2015

Multiplexing — many requests share one connection; binary framing; header compression.

HTTP/3

2022

Runs over QUIC (UDP-based) instead of TCP — faster connection setup, better on lossy networks.



HTTPS = HTTP + TLS encryption. Same request/response model — but wrapped in a TLS handshake that encrypts traffic and verifies the server's identity via a certificate. The padlock icon means "this connection is encrypted," not "this site is trustworthy."



LIVE DEMO

Speaking HTTP Directly With curl



```
$ curl -v https://example.com
> GET / HTTP/1.1           # the request we send
> Host: example.com
< HTTP/1.1 200 OK          # the response we get back
< Content-Type: text/html
< Content-Length: 1256
<html>...</html>
```



```
$ curl -I https://example.com      # HEAD request, headers only
$ curl -X POST -d 'name=Ana' https://httpbin.org/post  # send a POST
$ curl -o page.html https://example.com  # save response to a file
$ curl -w '%{http_code}\n' -o /dev/null -s https://example.com  # status only
```



LIVE DEMO

Reading Real Traffic in DevTools

- 1 Open any site, press F12 (or Cmd+Opt+I on Mac), go to the Network tab
- 2 Reload the page (Ctrl/Cmd + R) and watch requests populate
- 3 Click the main document request → Headers tab → compare to our slide above
- 4 Find a request that returned a non-200 status (try a deliberately broken image URL)
- 5 Filter by type: JS, CSS, Img, Fetch/XHR — point out how many separate requests one page makes
- 6 Right-click a request → Copy → Copy as curl — show it matches what we just typed manually



EXERCISE

Try It Yourself (5 minutes)

Using httpbin.org (a free HTTP testing service), run each command and predict the status code before you hit Enter:

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://httpbin.org/status/200
$ curl -s -o /dev/null -w '%{http_code}\n' https://httpbin.org/status/404
$ curl -s -o /dev/null -w '%{http_code}\n' https://httpbin.org/status/500
$ curl -s -L -o /dev/null -w '%{http_code}\n' https://httpbin.org/redirect/2
$ curl -s https://httpbin.org/headers      # see your own request headers echoed back
$ curl -s https://httpbin.org/get?name=Ana&city=Almaty  # see query params parsed
```

Web Servers

The program on the other end of every request

30 MINUTES





CONCEPT

What Is a Web Server, Really?

A web server is software that listens for HTTP requests on a port and sends back HTTP responses. The "server" is the program, not (just) the physical machine.

1 Listen

Bind to a port (80 for HTTP, 443 for HTTPS) and wait

2 Accept

A connection arrives from a client

3 Parse

Read the request line + headers + body

4 Handle

Decide what to do: read a file, run code, query a database

5 Respond

Send status line + headers + body back

6 Repeat

Stay listening for the next request (often handling many at once)



DISTINCTION

Static vs Dynamic Content



Static Content

Files sit on disk exactly as they'll be sent — HTML, CSS, JS, images.

Server's job: find the file, send it. Same request → same response, every time.

Fast, simple, very cacheable.

Examples: nginx serving a portfolio site, a CDN serving images.



Dynamic Content

Server runs code (PHP, Node.js, Python, Java...) to build the response on the fly.

Server's job: run a program, often query a database, assemble HTML or JSON, then send it.

Flexible, personalized.

Examples: your inbox, a logged-in dashboard, search results.



LANDSCAPE

Common Web Server Software



Nginx

High-performance, popular as a reverse proxy & load balancer



Apache HTTP Server

Long-running, highly configurable, huge module ecosystem



Node.js (Express, etc.)

JavaScript runtime — write the server logic yourself



Caddy

Automatic HTTPS, simple configuration

In production, a request often passes through several layers: **CDN** → **load balancer** → **reverse proxy (nginx)** → **application server (Node/Django/etc.)** → **database**. Today we focus on the core request/response model underneath all of that.



LIVE DEMO

Stand Up a Web Server in 60 Seconds

Step 1 — the simplest possible static server (built into Python):



```
$ mkdir site && cd site && echo '<h1>Hello, class!</h1>' > index.html
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 ... # now open http://localhost:8000
```

Step 2 — a tiny dynamic server (raw Node.js, no framework):



```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  console.log(req.method, req.url); // log every request
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end(`Hello! You requested: ${req.url}`); // dynamic body
});

server.listen(3000, () => console.log('Listening on :3000'));
```

```
$ node server.js # try /, /about, /anything - watch req.url change
```



Same Request, Two Different Servers

```
python3 -m http.server
```

GET /index.html

→ open ./index.html from disk → send its exact bytes back.

GET /missing.html

→ file not found → 404, automatically.

No code runs per request — purely file lookup.

```
node server.js
```

GET /anything

→ our function runs → we decide what to send, every single time.

There's no "404 automatically"

— if we don't write that logic ourselves, every path returns 200.

This is the core idea behind every backend framework.



Beyond Your Own Machine: Edge Platforms

You don't have to run `http.server` or `node server.js` on your own machine forever. Platforms like Cloudflare Pages & Workers run your static files and request-handling code for you, on servers distributed worldwide.



Cloudflare Pages

= our "`python3 -m http.server`" idea, but global.

Serves static sites: HTML/CSS/JS, built React/Vue apps, generated blogs.

Files are pushed to many edge locations — a visitor's request is answered by the nearest one.



Cloudflare Workers

= our "`node server.js`" idea, but global.

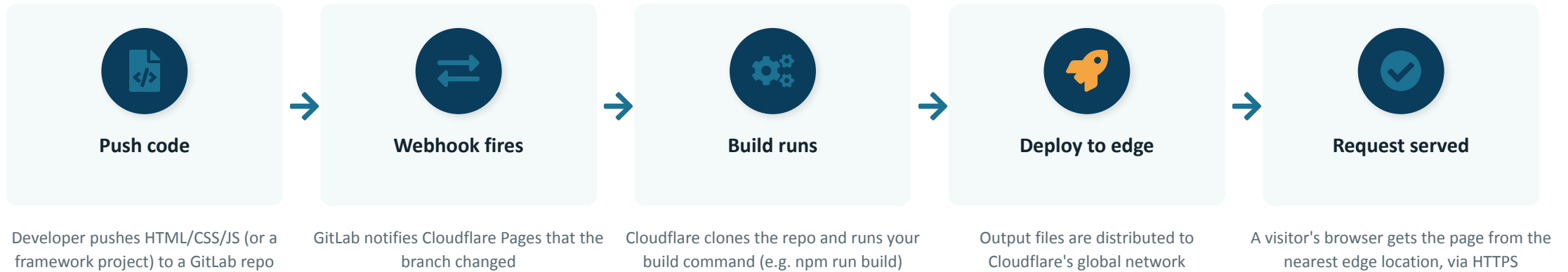
Runs your request-handling code (JS/TS, or compiled from other languages) at the edge — no server to provision or patch.

Same job as our Node example: receive a request, decide what to send back.



EXAMPLE

Deploying From a GitLab Repository



Why this matters for today's topic: everything we covered still applies — DNS still resolves your domain, it's still plain HTTP/HTTPS requests and responses, the browser still builds the DOM the same way. Only who runs the server, and where, has changed.

Web Browsers

The client that turns bytes into a page

30 MINUTES





CONCEPT

What Is a Browser, Really?

A browser is an HTTP client with a rendering engine attached. It does far more than just "display HTML":

1

Resolve the URL

Parses it, triggers DNS lookup if needed

2

Make the connection

TCP handshake, TLS handshake for HTTPS

3

Send the HTTP request

Including cookies, cache headers, etc.

4

Parse the response

Read headers, decide how to interpret the body

5

Build the DOM

Parse HTML into a tree of objects

6

Fetch sub-resources

CSS, JS, images, fonts — each its own request

7

Render

Compute layout (CSSOM) and paint pixels to the screen

8

Execute JavaScript

Can modify the DOM, fire more requests, handle events



Browser Engines You Should Recognize

Blink

Chrome, Edge, Opera, Brave

Rendering engine forked from WebKit; V8 JavaScript engine

Gecko

Firefox

Mozilla's engine; SpiderMonkey JavaScript engine

WebKit

Safari (and all iOS browsers)

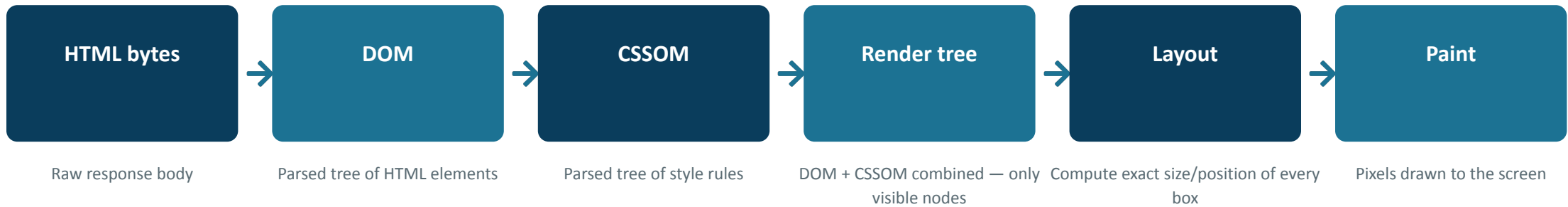
Apple's engine; JavaScriptCore engine

Why it matters: rendering quirks and JS feature support can differ slightly between engines — hence "cross-browser testing."



PIPELINE

From Bytes to Pixels



Note: JavaScript can re-enter this pipeline at any point — modifying the DOM triggers re-layout and re-paint. This is why heavy JS manipulation can hurt performance, and why frameworks batch updates.



ESSENTIAL SKILL

Your Most Important Debugging Tool

Elements

Live DOM + CSSOM — inspect and edit on the fly

Console

Run JS, see errors/warnings/logs

Network

Every request: headers, timing, status, payload

Application

Cookies, local storage, cache, service workers

Performance

Record and analyze the rendering pipeline

Sources

Inspect/debug JS with breakpoints

F12 (Windows/Linux) · **Cmd + Option + I (Mac)** · **Right-click → Inspect**



LIVE DEMO

Watching the Whole Pipeline Happen

- 1 Elements tab: edit a heading's text live, change a CSS color — see it update instantly (not on the server!)
- 2 Console: type `document.title`, then `document.body.style.background = 'pink'`
- 3 Network tab → throttle to 'Slow 3G' → reload → watch the waterfall of requests load in order
- 4 Network tab → check 'Disable cache' vs not — reload twice, compare timings
- 5 Application tab → Cookies: show a real Set-Cookie value sitting in storage
- 6 Performance tab → Record → reload → stop → show the parse/layout/paint breakdown

Putting It All Together

One URL, every concept from today, start to finish

15 MINUTES





SYNTHESIS

Typing a URL: The Complete Journey



1. You type

`https://shop.example.com/cart` and press Enter



2. DNS resolution

Browser → resolver → root → .com → authoritative NS → IP address



3. Connection

TCP handshake to that IP on port 443, then TLS handshake (certificate check)



4. HTTP request

Browser sends `GET /cart HTTP/1.1` with headers and cookies



5. Server processing

Web server runs code, maybe queries a database, builds a response



6. HTTP response

Status 200, headers, HTML body sent back over the same connection



7. Rendering

Browser parses HTML → DOM, fetches CSS/JS/images (more requests!), builds render tree



8. Paint

Pixels appear on your screen — total elapsed time: usually under a second



IMPORTANT

A Brief Security Note



HTTPS is the baseline, not optional

Every production site should use it — browsers actively flag plain HTTP as "Not Secure."



The padlock confirms encryption, not trust

It means traffic can't be easily read in transit. It says nothing about whether the site itself is legitimate.



DNS responses can be spoofed

DNSSEC exists to cryptographically verify DNS answers — worth knowing the name, even if we won't implement it today.



Never trust client-supplied data

Because HTTP is just text anyone can write, a server must always validate input — never assume the browser "protected" it for you.



RECAP

Key Takeaways

- 1 The Internet is infrastructure; the Web is one application on top of it
- 2 DNS translates human-friendly names into IP addresses, through a hierarchy of servers, with caching governed by TTL
- 3 HTTP is a stateless request/response protocol — verbs, status codes, and headers are its vocabulary
- 4 A web server is just a program that listens, parses requests, and sends responses — static or dynamic
- 5 A browser is an HTTP client plus a rendering engine: HTML → DOM → render tree → layout → paint
- 6 DevTools' Network tab is the single most useful tool for understanding (and debugging) all of the above



HOMEWORK

Lab Assignment — Due Next Class

1. DNS report

Run `dig` (or `nslookup`) on 3 domains of your choice. Record their A record, TTL, and authoritative nameservers. Note how long each TTL is and guess why.

2. curl scavenger hunt

Find one URL that returns each of: 200, 301 or 302, 404, and 500/503. Submit the `curl` commands and the status codes you got.

3. Build & inspect

Write the 10-line Node.js server from class, add a second route, and submit a screenshot of its request/response in the DevTools Network tab.

4. Short written answer (½ page)

Pick one real website. Open DevTools → Network. Describe 3 different requests it made beyond the main HTML document, and what each one was for.



RESOURCES

Further Reading



MDN Web Docs — HTTP

developer.mozilla.org/en-US/docs/Web/HTTP

The single best HTTP reference, including every header and status code



High Performance Browser Networking

hpbn.co (free online book by Ilya Grigorik)

Deep dive into TCP, TLS, HTTP/2, and browser networking



How DNS Works (comic)

howdns.works

A fun, visual explainer of the DNS resolution process



"What happens when you type a URL" (GitHub gist)

Search: [alex/what-happens-when](#)

A widely-shared, exhaustive version of today's final synthesis slide



Questions?

Next lecture: HTML & the Document Object Model

Bring a laptop with a browser and terminal access — we'll write our first real HTML documents from scratch.